CHAPTER 13

Program Development with Classes

> I paint objects as I think them, not as I see them.
>
> Pablo Picasso, artist

IN THIS CHAPTER, WE WILL DEVELOP A MORE IN-DEPTH EXAMPLE USING CLASSES SO you can see how problem-solving design can be accomplished using classes (objects).

13.1 PREDATOR–PREY PROBLEM

For this problem, we will consider a simulation of a natural habitat: a predator–prey problem. Predator–prey problems are a study of the varying sizes of populations as two groups of animals interact. One group constitutes the prey, a population of animals that are the food source for the other population of animals, the predators. The simulation shows the dynamic interaction of these two populations. It represents a kind of habitat war as the two populations struggle to survive or thrive.

Typically, both groups have a fixed birthrate. The prey usually procreate faster than the predators, allowing for a growing prey population. However, as the population of prey increases, the habitat can support a higher number of predators. This in turn leads to an increasing predator population and, after some time, a decreasing prey population. Around that time the population of predator grows so large as to reach a critical point, the point where the number of prey can no longer support the present predator population and the predator population begins to wane. As the predator population declines, the prey population recovers and the two populations continue this interesting interaction of growth

and decay. Though predator–prey relationships have been modeled with equations[1] it is interesting to simulate the relationship on a computer and observe how they interact. The classic simulation may be the WAT-OR game, based on a "Scientific American" article by A. K. Dewdney.[2] We choose a slightly different model here.

As with any simulation, the programmer gets to choose what is represented, as no reasonably sized computer program could encompass the complexity of a full habitat. Within such a simulation, the user can adjust the identified parameters, such as birth and death rates of each population, and observe the impact on each cycle, and each population.

An actual example of studying predator–prey relationships in their habitat is the one between wolves and moose on Isle Royale in Lake Superior, as shown at the website `http://www.isleroyalewolf.org`. Isle Royale is located in the northwest of Lake Superior and its population of wolves and moose are isolated to the island (occasionally, Lake Superior does freeze enough for walking to land). Its isolation is a perfect laboratory for studying the effects of predator–prey interaction. A graph of the varying population sizes as actually has occurred since 1959 can be found at: `http://www.isleroyalewolf.org/data/data/home.html`.

13.1.1 The Rules

These are the rules we will use for our simulation:

- The habitat updates itself in units of time called clock *ticks*. During one tick of our habitat clock, every animal in the island gets an opportunity to do something.
- All animals are given an opportunity to move into an adjacent space, if an empty adjacent space can be found. One move per clock tick is allowed.
- Both the predators and prey can reproduce. Each animal is assigned a fixed *breed time*. If the animal is still alive after breed-time ticks of the clock, it will reproduce. The animal does so by finding an unoccupied, adjacent space and fills that space with the new animal—its offspring. The animal's breed time is then reset to zero. An animal can breed at most once in a clock tick.
- The predators must eat. They have a fixed *starve time*. If they cannot find a prey to eat before starve-time ticks of the clock, they die.
- When a predator eats, it moves into an adjacent space that is occupied by prey (its meal). The prey is removed and the predator's starve time is reset to zero. Eating counts as the predator's move during that clock tick.
- At the end of every clock tick, each animal's local event clocks is updated. All animals' breed times are decremented and all predators' starve times are decremented.

This is the model we will need to support. It doesn't perfectly model the actual environment, but it does sufficiently well to provide insight into predator–prey interactions.

---

[1] Lotka–Volterra equations.

[2] *Scientific American*, December Issue, 1984.

13.1.2 Simulation Using Object-Oriented Programming

Simulation is perfectly suited for object-oriented programming (OOP). In OOP, we need to identify objects and define their interactions to create a program. Simulations naturally require these kind of identifications. It is for this reason that OOP evolved out of the programming language Simula, a programming language for writing simulations.

What objects do we need for our simulation? We need an island, some moose (the prey), and a few wolves (the predators). We also need all the methods that govern how they interact with each other, as well as a main program. Let's focus on the classes we need for the moment. We need:

- `Island` Class
- `Prey` Class
- `Predator` Class

Having a single `Prey` class and a single `Predator` class, we can create many instances of these classes and have the instances interact. Very efficient!

We now need to identify for each class:

- *What* we need to have represented in each class instance.
- *How* these instances can be manipulated, that is, what methods are needed.

Next, look at each class and try to sketch out what is required.

13.2 CLASSES

13.2.1 *Island* Class

Isle Royale is shaped like an elongated rectangle, but to keep our example simple, we choose to model an `Island` instance as a square, an $n \times n$ grid. Every element in the grid could potentially have *one* of three values. Either it contains nothing, a predator or a prey.

To begin with, the grid is initialized to be empty at every grid position. The value 0 will be used to represent an empty grid position. Every class needs a constructor (`__init__`) and it is generally useful to have a `__str__` method so that we can easily display an `Island` instance. When we create an instance, we need to specify only its size—in our case, we'll specify the length of a side since the island is square. This will be an argument to our constructor. See Code Listing 13.1.

Code Listing 13.1

```
class Island():
    """Island
       n X n grid where zero value indicates an unoccupied cell."""
```

```
    def __init__(self, n):
        '''Initialize cell to all 0's, then fill with animals
        '''
        self.grid_size = n
        self.grid = []
        for i in range(n):
            row = [0]*n    # row is a list of n zeros
            self.grid.append(row)

    def __str__(self):
        '''String representation for printing.
           (0,0) will be in the lower-left corner.
        '''
        s = ""
        for j in range(self.grid_size-1,-1,-1):  # print row size-1 first
            for i in range(self.grid_size):      # each row starts at 0
                if not self.grid[i][j]:
                    # print a '.' for an empty space
                    s+= "{:<2s}".format('.' + "  ")
                else:
                    s+= "{:<2s}".format((str(self.grid[i][j])) + "  ")
            s+="\n"
        return s
```

```
In  [1]: from island1 import *
In  [2]: royale = Island(10)
In  [3]: print(royale)
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
```

In the `__init__` constructor, we save the size of the instance and the grid that represents the instance. Those two items are sufficient to represent the present state of the `Island` instance—essentially, the location in the grid of all the predators and prey.

A grid of n rows and n columns can be thought of as a list of n rows—each of length n. We create a row that is a list of n items of value 0 using the repeat (*) operator. We then add n rows to our instance grid, making the $n \times n$ grid.

The `__str__` method prints the contents of each grid element. To keep with standard Cartesian coordinates, we have the method print in such a way that (0,0) is in the lower-left corner. This means we have x increasing to the right and y increasing to the top. However, because printing occurs from the top down, we must print the highest-numbered row first.

The output in the previous session shows the creation of a 10×10 `Island` instance named "royale" followed by printing of the current state of the instance: empty.

13.2.2 Predator and Prey, Kinds of Animals

At this point, it is worth our time to think a little about the rest of our simulation design. If we look at the requirements for the two kinds of animals in our simulation, predator and prey, we can see that they share some important characteristics. They occupy locations on the `Island` instance, and they must be able to move and breed. Because they share these characteristics, it makes sense that they *inherit* those characteristics from a common parent, the `Animal` object.

`Animal` Object

What state does each animal instance need? Each instance needs to know where it is on the `Island` instance. The question is: where do we keep that location information—with the `Animal` instance, with the `Island` instance, or with both? Each `Animal` instance needs to know where it is, its local information, but it also needs to be aware of what else is in its surroundings, the global information. In addition, we will probably want to control how many animals can be at the same place at the same time. For those reasons, an `Animal` instance will have as *local* state its location on the island grid, its x and y coordinates. We let the `Island` instance keep *global* state information on all the inhabitants by marking in each grid location what is presently there, including nothing.

Therefore, our `Animal` object will need the following:

- Its name (string, e.g., `'moose'` or `'wolf'`).
- Its location: (x, y) coordinates on the island.
- The `Island` instance, that is, the island the animal is on.

The `__init__` constructor will need to take in each of these values as parameters. The x and y values indicate where on the island the animal will be placed. The `Animal` instance will also need to know the island it is located on. Finally, the instance will need its name: moose and wolf.

We need to inform the `Island` instance when a new `Animal` instance is added to the island. For that purpose we create the `register` method in the `Island` class. This method will "place" (register) the `Animal` instance in the `Island` grid at the x and y coordinates stored in the instance. In this way, we coordinate the local information (x and y coordinates stored in the `Animal` instance) with the global information (location of each instance in the `Island` instance grid) for the simulation.

Also, each `Animal` instance will need to know how big the `Island` instance is so that it can move around without falling into the surrounding water. For that purpose, we will also create a `size` method in the `Island` class.

Our communication between the animal and the island will be:

- `n = island.size()  # get the size from the island instance`
- `island.register(self)  # register an animal (self) location with the island instance`

Let's add those two methods to the `Island` class. Also, let's create the bare bones of the `Animal` class so that we can begin to work with the interaction between animals and the island. This is shown in Code Listing 13.2.

Code Listing 13.2

```
class Island (object):
    """Island
       n X n grid where zero value indicates not occupied."""
    def __init__(self, n, prey_count=0, predator_count=0):
        '''Initialize grid to all 0's, then fill with animals
        '''
        print n,prey_count,predator_count
        self.grid_size = n
        self.grid = []
        for i in range(n):
            row = [0]*n    # row is a list of n zeros
            self.grid.append(row)
        self.init_animals(prey_count,predator_count)

    def size(self):
        '''Return size of the island: one dimension.
        '''
        return self.grid_size

    def register(self,animal):
        '''Register animal with island, i.e. put it at the
        animal's coordinates
        '''
        x = animal.x
        y = animal.y
        self.grid[x][y] = animal

    def __str__(self):
        '''String representation for printing.
           (0,0) will be in the lower left corner.
        '''
```

```
        s = ""
        for j in range(self.grid_size-1,-1,-1):  # print row size-1 first
            for i in range(self.grid_size):      # each row starts at 0
                if not self.grid[i][j]:
                    # print a '.' for an empty space
                    s+= "{:<2s}".format('.' + "  ")
                else:
                    s+= "{:<2s}".format((str(self.grid[i][j])) + "  ")
            s+="\n"
        return s

class Animal(object):
    def __init__(self, island, x=0, y=0, s="A"):
        '''Initialize the animal's and their positions
        '''
        self.island = island
        self.name = s
        self.x = x
        self.y = y

    def __str__(self):
        return self.name
```

```
In  [1]: royale = Island(10)
In  [2]: animal1 = Animal(island=royale,x=4,y=8,s='a1')
In  [3]: animal2 = Animal(island=royale,x=6,y=4,s='a2')
In  [4]: royale.register(animal1)
In  [4]: royale.register(animal2)
In  [4]: print(royale)
.   .   .   .   .   .   .   .   .   .
.   .   .   .   a1  .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   a2  .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .
In  [5]: print(animal1)
a1
```

Notice how the printing of an `Island` is accomplished in its `__str__` method. We access the grid with *`self`*`.island[i][j]`, but what is located there? Examine the `Island` `register` method. It is passed an `Animal` instance as a parameter. It is this instance value

that is stored in the grid. Thus *self*.island[i][j] is a reference to that Animal instance, and because that call is within __str__, it is asking for the string representation of that particular animal. That is found in the Animal's __str__ method. At the end of the session, we show an explicit call to the Animal's __str__ method by way of a ***print*** statement.

For debugging purposes, it would be nice to confirm that the Animal instance grid coordinates agree with the island grid location. Printing the Island instance indicates the location that instance is maintaining for each Animal instance. The opposite, accessing position information for an Animal instance, requires a new Animal method. Let's call it position, and we will have it simply return the Animal instance's (x, y) coordinate pair. Code Listing 13.3 shows the method (without the rest of the Animal class). We can see in the accompanying session that both the Island instance and the Animal instance have the same location coordinates for that Animal instance.

Code Listing 13.3

```
def position(self):
    """Return coordinates of current position."""
    return self.x, self.y
```

```
In  [1]: royale = Island(10)
In  [2]: animal1 = Animal(island=royale,x=0,y=8,s='a1')
In  [3]: animal1.position()
Out [3]: (0, 8)

In  [4]: print(royale)
.  .  .  .  .  .  .  .  .  .
a1 .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
.  .  .  .  .  .  .  .  .  .
```

13.2.3 Predator and Prey Classes

The Predator and Prey classes share many characteristics, which are collected in the Animal class. We can take advantage of this sharing by having each be a subclass of

Animal. Each Predator and Prey class will require particular methods, but they can also share many methods and variables by being subclasses of the Animal class.

In particular, remember that the __init__ method of the subclass must call the __init__ of the parent class to fully initialize the instance. Code Listing 13.4 shows the two new classes with just their respective __init__ methods. Note that we do not need to define a __str__ method for either subclass, as this method already exists in the Animal class.

Code Listing 13.4

```
class Animal(object):
    def __init__(self, island, x=0, y=0, s="A"):
        '''Initialize the animals and their positions
        '''
        self.island = island
        self.name = s
        self.x = x
        self.y = y

    def __str__(self):
        return self.name

class Prey(Animal):
    def __init__(self, island, x=0,y=0,s="O"):
        Animal.__init__(self,island,x,y,s)

class Predator(Animal):
    def __init__(self, island, x=0,y=0,s="X"):
        Animal.__init__(self,island,x,y,s)
```

13.2.4 Object Diagram

It is useful to create a diagram of the objects we have created to help understand the relationships among them. See Figure 13.1.

13.2.5 Filling the Island



VideoNote 13.1 Improve Simulation

Now that we have the Prey, Predator, and Island classes, we need a way to fill an Island instance with instances of both Predator and Prey. We could leave this as a separate function to be written as part of the main program, but it makes sense that the Island instance should be able to fill itself given some counts for each class.

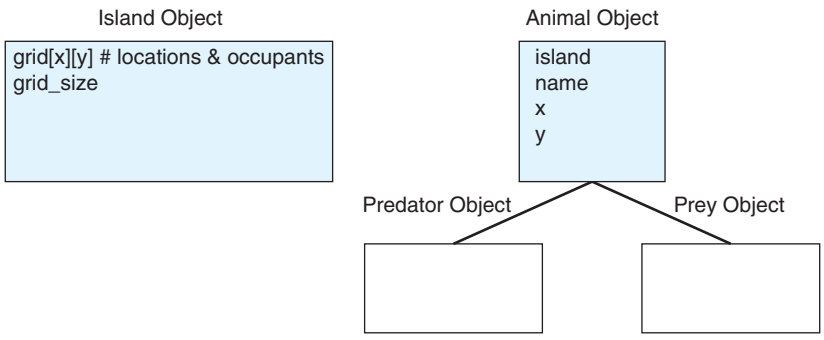


FIGURE 13.1 Objects: island, animals, predators, and prey.

To accomplish this we add the `init_animals` method of `Island`. It takes two parameters, the predator and prey counts, and fills the island with the correct number of instances of these two classes.

We would like to place the animal instances at random spots on the `Island` instance. That means that we will need to import the `random` module and use the `randint` method to get the x and y coordinates to place an animal. The value generated will be between 0 and the island's size. See Section 16.1.5 for more information on the `random` module.

The generated values must place the new instance at an unoccupied grid location. If the present (x, y) is occupied, another pair must be generated.

Querying a Grid Location

We need some way to query an `Island` instance to see if an `Animal` instance is at a location. To do that, the island needs a method to respond to that query. Let's call that method `animal()`. It will take the x and y coordinates as arguments:

```
def animal(self,x,y):
    if 0 <= x < self.size   and   0 <= y < self.size:
        return   self.island[x][y]
    else:
        return   -1 # outside island boundary
```

The method returns an instance (if one occupies the position), a 0 (if the position is empty), or a -1 to indicate that the coordinates are off the island (explained later).

Repeat Until Full

With these pieces in hand, we can do the following to fill the island:

- Generate an (x, y) pair using `random.randint(0,grid_size)`
- Check to see whether that position is empty or has an animal instance using the `animal` method

- If the position is empty:
  - Create the new instance, record the *x* and *y* values in the instance, then register the instance.
  - Increment the count of created instances. Initialization of that animal (predator or prey) ends when the count reaches the goal count for the animal (passed as an argument to the constructor).
- If the position is occupied:
  - The **`while`** loop continues, but the count of created instances is not incremented.

If the `Island` instance is very full, this process could be rather inefficient. Perhaps you could improve it? (See the Exercises.)

The remaining detail of this section is to make two small changes to the `Island` `__init__` method. First, two more parameters are passed to `Island`: the counts for the `Predator` and `Prey` instances. They default to 0, if not specified. Second, a line is added to the end of the constructor to call the `init_animals` method. The `Island` class with these additions is shown in Code Listing 13.5, but to avoid clutter we have omitted the unmodified methods: `size`, `register`, and `__str__`.

Code Listing 13.5

```
class Island (object):
    def __init__(self, n, prey_count=0, predator_count=0):
        '''Initialize grid to all 0s, then fill with animals
        '''
        # print n,prey_count,predator_count
        self.grid_size = n
        self.grid = []
        for i in range(n):
            row = [0]*n    # row is a list of n zeros
            self.grid.append(row)
        self.init_animals(prey_count,predator_count)

    def animal(self,x,y):
        '''Return animal at location (x,y)'''
        if 0 <= x < self.grid_size and 0 <= y < self.grid_size:
            return self.grid[x][y]
        else:
            return -1  # outside island boundary

    def init_animals(self,prey_count, predator_count):
        ''' Put some initial animals on the island
        '''
        count = 0
```

```
        # while loop continues until prey_count unoccupied positions are found
        while count < prey_count:
            x = random.randint(0,self.grid_size-1)
            y = random.randint(0,self.grid_size-1)
            if not self.animal(x,y):
                new_prey=Prey(island=self,x=x,y=y)
                count += 1
                self.register(new_prey)
        count = 0
        # same while loop but for predator_count
        while count < predator_count:
            x = random.randint(0,self.grid_size-1)
            y = random.randint(0,self.grid_size-1)
            if not self.animal(x,y):
                new_predator=Predator(island=self,x=x,y=y)
                count += 1
                self.register(new_predator)

## Unmodified methods not shown: size, register, __str__
```

13.3 ADDING BEHAVIOR

We have a start with our simulation. We have an `Island` instance on which we can place `Predator` and `Prey` instances. Now we need to have these animals do something—let's begin with movement.

13.3.1 Refinement: Add Movement

Neither the `Predators` nor `Prey` instances should stay still, so let's add movement. Immense complexity of movement can be modeled, but we will begin with simple "random" movement (truly random movement will added later). Because both kinds of instance needs to move, we will add a new method to `Animal`.

The movement we allow is to move to a neighboring region, if it is empty. If no neighboring locations are empty, the animal instance cannot move and remains at its present location.

Our algorithm to find a location to move to is as follows:

1. for each neighboring location:
   (a) if neighboring location is empty:
      i. move to neighboring location
      ii. break (stop looking for a place to move to)

What is a "neighboring region"? Any square not only shares edges with four neighbors, but also shares a corner with another four neighbors. See Figure 13.2. A reasonable argument can be made for moving only to the four neighboring regions sharing edges. However, let's allow these animals more freedom of movement and let them jump to any of the eight neighboring regions.

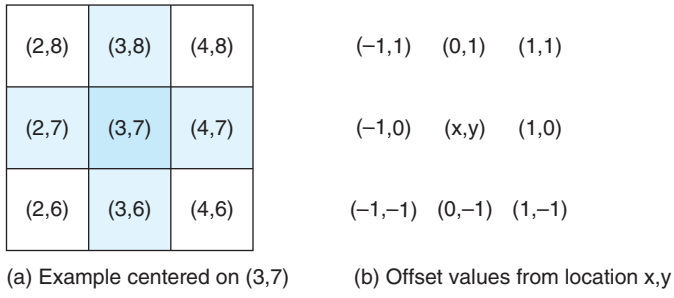


FIGURE 13.2 Eight neighbors.

If an animal is at a particular location (particular coordinates), what are the coordinates of the eight neighboring locations that it might move to? Figure 13.2(b) shows the offsets of the eight neighbors. We can add the offset to an animal's current coordinates to find the coordinates of a neighbor. For example, if an animal is at (3,7), the location immediately below it has offset (0,–1) so the neighboring coordinates are (3+0,7–1) = (3,6). See Figure 13.2(a).

To implement movement, we can loop through the neighbors, but what is the best way to represent neighbors for looping? The offsets provide regularity that can be worked into a loop. Let's make a list of offsets and then refer to the offset list each time we want to determine coordinates for a neighbor. Examining each neighbor becomes a walk through the offset list, applying each offset to an animal's current location. A refinement of our algorithm now looks like this:

1. for each offset:
   (a) neighbor_x = current_x + offset_x
   (b) neighbor_y = current_y + offset_y
   (c) if neighboring location is empty:
      i. move to neighboring location
      ii. break (stop looking for a place to move to)

We now have two new actions to consider. First, how do we know if a neighbor is empty? We can use the `animal` method previously defined for the `Island` class. That method returns an `Predator` or `Prey` instance, if it exists at the specified location.

Second, how do we move to a neighboring location? We can update the local location, the *x* and *y* coordinates in the animal instance, but we also need to update the island to keep the global information in synch. We also need to remove the animal from its current location and then register it in its new location. Therefore, the `Island` class needs a method to remove an animal from a given location. Removal simply sets the grid location to 0.

```
def remove(self,x,y):
    self.grid[x][y] = 0
```

With those two island methods in hand, we can write the `Prey's move` method and test it. This is shown in Code Listing 13.6.

Code Listing 13.6

```
def move(self):
    """Move to an open, neighboring position."""
    # neighbor offsets
    offset = [(-1,1),(0,1),(1,1),(-1,0),(1,0),(-1,-1),(0,-1),(1,-1)]
    for i in range(len(offset)):
        x = self.x + offset[i][0]  # neighboring coordinates
        y = self.y + offset[i][1]
        if self.island.animal(x,y) == 0: # neighboring spot is open
            self.island.remove(self)  # remove from current spot
            self.x = x  # new coordinates
            self.y = y
            self.island.register(self) # register new coordinates
            break  # finished with move
```

```
In [1]: royale = Island(5) # create island
In [2]: moose1 = Prey(royale,"m1") # create two moose
In [3]: moose2 = Prey(royale,"m2")
In [4]: print(royale) # print initial state of island
.  .  .  .  .
.  .  .  .  .
.  .  m1 .  .
.  m2 .  .  .
.  .  .  .  .

In [5]: moose1.move() # move each moose
In [5]: moose2.move()
```

```
In [5]: print(royale) # observe island after moves
.   .   .   .   .
.   m1  .   .   .
m2  .   .   .   .
.   .   .   .   .
.   .   .   .   .
```

Note how each neighboring coordinate is calculated using `offset`. Each item in the `offset` list is a tuple: `(x,y)`. To get the *x* value of offset "i" we use `offset[i][0]`, that is, the first item of the pair. The *y* value is similar. Also, note how the first offset is up-and-left so each animal in the test moves up one and left one position.

Falling off the Island?

Some of you may be asking, "How do we keep from falling off the island?" It is a good question. Some models of a "world" allow the world to "wrap" at the edges. That is, if you walk off the island going left (an *x* value less that 0), you reset yourself to the right (to an *x* value of 9, or whatever the size of the island is). Other approaches simply prevent you from making a "bad" move.

Our particular approach is the latter: we prevent a bad move. This prevention is done in the `animal` method of `Island`. If the passed arguments are out of range, the method returns a −1 indicating the problem.

Is this a good approach? See the exercises for other options.

13.3.2 Refinement: Time Simulation Loop

Let's add a `main` function to initialize our world and a loop to drive our simulation. We add a number of each of the `Predator` and `Prey` instances to an `Island` instance. Then, in the main loop, each trip through the loop constitutes one unit of time. In that unit of time, we give each instance the opportunity to move. We do this by moving through each grid element in the `Island` instance and, if the element holds an animal, allow it to move.

Here is an outline of our driving loop:

1. Create `Island` instance with a set number of `Predator` and `Prey` instances.
2. Loop for a specified number of time ticks:
   (a) Move each instance.

Let's take a look in Code Listing 13.7 at how that driving loop might be implemented in Python. We included a ***`print`*** statement so we can observe the movement in each time step. However, we only print the first two time steps here.

Code Listing 13.7

```
def main ():
    # initialization of the simulation
    royale = Island(5,1,1) # 5x5 island, 1 predator, 1 prey
    time_steps = 20

    # run the event loop
    island_size = royale.size()
    count = 0
    while count < time_steps:
        print(royale) # print the island
        for x in range(island_size):
            for y in range(island_size):
                animal = royale.animal(x,y)
                if animal:
                    animal.move()
        count += 1
```

```
.   m   .   .   .
.   .   .   w   .
.   .   .   .   .
.   .   .   .   .
.   .   .   .   .

m   .   .   w   .
.   .   .   .   .
.   .   .   .   .
.   .   .   .   .
.   .   .   .   .
< rest of simulation not shown >
```

Notice again how animals tend to move up and to the left. That is because our `move` method always works through the offsets in the same order. Improving on that movement is left as an exercise.

13.4 REFINEMENT: EATING, BREEDING, AND KEEPING TIME

The next refinements introduce a number of complications that require close attention. First, we need to add breeding to the `Animal` class, allowing both `Predator` and `Prey` instances to breed. Second, we differentiate `Prey` and `Predator` classes by introducing

the eating behavior to the `Predator` class. In our simple model only predators eat and, if they cannot eat, they starve. Third, the time span in which prey and predators breed must be different. For the simulation to work, prey must breed at a faster rate than predators—otherwise, predators quickly starve. Finally, yet another time span must be introduced in which predators must eat or, if they cannot, starve.

The ability to set these time spans is important as their values are critical for the simulation. If `Prey` instances cannot breed quickly enough, then they will *all* be consumed, ending the simulation. If not enough `Prey` instances are provided, the `Predator` instances will *all* starve and the simulation will end. Part of the interest in the simulation is examining what values will work and how they affect the simulation. We must therefore find a way for the user to easily changes these values.

13.4.1 Improved Time Loop

We can start by improving the main function, both for initialization of values and for introducing the new (yet unwritten) behaviors into the main loop. The new main function will now take seven parameters to establish the various aspects of the simulation. Those parameters are:

`predator_breed_time`: the time span that must pass before a `Predator` instance can breed.
`predator_starve_time`: the time span that a `Predator` instance must eat within, otherwise it will starve.
`initial_predators`: the number of `Predator` instances that will initially be placed on the Island.
`prey_breed_time`: the time span that must pass before a `Prey` instance can breed.
`initial_prey`: the number of `Prey` instances that will initially be placed on the island.
`size`: the size of a single side of the `Island` instance (which, you remember, is a square).
`ticks`: the number of time "ticks" that will pass before the simulation ends.

We can also provide default values for these parameters that constitute a runnable version of the simulation. However, the user is now free to change them by simply indicating which ones to override when calling the new main function.

Remembering the Breed and Starve Time Spans

Where can we place the breeding and starving time spans (`predator_breed_time`, `prey_breed_time`, `predator_starve_time`)? They are set as variables in the main function (as parameters), but this does not make them available to each individual prey and predator instance. One option would be to set the values outside of the main function as global variables, but global variables are undesirable, for a number of reasons. Another option would be to pass them as arguments when we create `Predator` and `Prey`

instances—adding another parameter to the classes. That is a good option, but let's choose yet another option that shows off some of Python's object-oriented capabilities.

Remember how a variable is searched for in a class hierarchy (see Section 12.6). First Python looks in the instance, then in the class, then up the class hierarchy through the class's parent class. If we place a variable in both the predator and prey *classes* (not the instance, the class), that value will be globally available to all instances of the two classes. A potential issue is that if we later change the value of that variable, it is changed for all instances, so we must be careful! That issue may be desirable or undesirable, depending on our needs. In our case, we will use these values only in the constructor (`__init__`) so any changes to the values will impact instances as they are created. We create three variables in the `main` function with the initial parameter values: `Predator.breed_time`, `Predator.starve_time`, and `Prey.breed_time`. Because each specifies a class name (`Predator` or `Prey`), these assignments augment the respective class definitions; that is, these names are added to the class namespace. These values will now be available for all the instances of those classes when we create them.

We then extend the event loop to include eating and breeding (for `Predator` instances only in the listing). We also include at the end of the event loop a new method, `clock_tick`. This method will "change" each instance based on its type. Anything that must happen every time tick for an instance is handled here. The updated driver is shown in Code Listing 13.8.

Code Listing 13.8

```
def main(predator_breed_time=6, predator_starve_time=3, initial_predators=10,\
         prey_breed_time=3, initial_prey=50, size=10, ticks=300):
    ''' main simulation; sets defaults, runs event loop, plots at the end
    '''
    # initialization values
    Predator.breed_time = predator_breed_time
    Predator.starve_time = predator_starve_time
    Prey.breed_time = prey_breed_time

    # make an island
    isle = Island(size,initial_prey, initial_predators)
    print(isle)

    # event loop.
    # For all the ticks, for every x,y location.
    # If there is an animal there, try eat, move, breed and clock_tick
    for i in range(ticks):
        for x in range(size):
            for y in range(size):
                animal = isle.animal(x,y)
                if animal:
                    if isinstance(animal,Predator):
```

```
            animal.eat()
        animal.move()
        animal.breed()
        animal.clock_tick()
```

Updating the `__init__` Methods

We need to include these new time spans in the instance. Each instance will have its own clock to keep track of its breeding time. The `Predator` instance will have a second clock to keep track of its starvation time. That is, when an instance is created its internal clocks need to be initialized. Both `Predator` and `Prey` instances should be initialized with a `breed_clock` whose value is set to the respective class `breed_time` value. Each instance will then update its clock during every clock tick via that `clock_tick` method. When their individual breeding clocks hit 0, they may breed. Similarly, `Predator` instances should be initialized with a `starve_clock`, initialized to the value in `Predator.starve_time`.

The updated methods are shown in Code Listing 13.9.

Code Listing 13.9

```
class Prey(Animal):
    def __init__(self, island, x=0,y=0,s="O"):
        Animal.__init__(self,island,x,y,s)
        self.breed_clock = self.breed_time

class Predator(Animal):
    def __init__(self, island, x=0,y=0,s="X"):
        Animal.__init__(self,island,x,y,s)
        self.starve_clock = self.starve_time
        self.breed_clock = self.breed_time
```

Where did `self.breed_time` and `self.starve_time` get set? They are not assigned in the `Predator` or `Prey` class definitions. As mentioned earlier, they were added to the class namespace in the `main` function where we assigned `Predator.breed_time`, `Predator.starve_time`, and `Prey.breed_time`. When we assigned those values, they were added to the respective classes' namespace, so they were available when the constructors were called. Notice how in the `Predator` and `Prey` classes the `self` in `self.breed_time` was resolved to the respective class name (`Predator` or `Prey`) when in the `main` function we assigned values to `Predator.breed_time` and `Prey.breed_time`. The starve time worked similarly.

Taking this approach with the `breed_time` and `starve_time` shows how class definitions can be dynamically augmented in Python—a powerful capability, but one that must be used carefully to ensure that readability is maintained.

13.4.2 Breeding

The breeding method is common to both subclasses (except for their time spans, as described in the previous section) so we place the `breed` method in `Animal`.

The rules described for the simulation said that when an animal breeds, the local neighborhood is examined for the first unoccupied location. If such a location is found, the new instance (the offspring) is placed at that location. It is unspecified what to do if an unoccupied location cannot be found—something we will have to resolve.

The "Neighbor" Method and Updating Move

Again, a little introspection is useful here. The process of looking for an unoccupied space used for breeding looks quite like the `move` method already described, which also requires an unoccupied neighbor. In fact, when we get to the `eat` method we will need to do something similar again, though in the eating case it will be to look for a neighbor that is `Prey`, not an unoccupied neighbor. It seems that we might be able to abstract out the "check neighbors" function and use it in many places.

We separate the "check neighbors" method from the existing `move` method so we can abstract the neighbor-checking aspect. That is, we *refactor* the move code! We call the new method `check_grid`, also part of the `Animal` class. It checks the eight neighbors for a location and returns either the location or 0.

One change is that we add a new parameter to `check_grid`, the `type_looking_for` parameter. Instead of looking only for empty grid locations, we look for a location that contains the `type_looking_for` type. If we want an unoccupied location, we are looking for a grid location with type `int` since an unoccupied location has a 0 value. However, we could also look for a location that has a `Prey` instance (for eating).

The updated move code and added `check_grid` are shown in Code Listing 13.10.

Code Listing 13.10

```
def check_grid(self,type_looking_for=int):
    ''' Look in the 8 directions from the animal's location
    and return the first location that presently has an object
    of the specified type. Return 0 if no such location exists
    '''
    # neighbor offsets
```

```
        offset = [(-1,1),(0,1),(1,1),(-1,0),(1,0),(-1,-1),(0,-1),(1,-1)]
        result = 0
        for i in range(len(offset)):
            x = self.x + offset[i][0]  # neighboring coordinates
            y = self.y + offset[i][1]
            if not 0 <= x < self.island.size() or \
               not 0 <= y < self.island.size():
                continue
            if type(self.island.animal(x,y))==type_looking_for:
                result=(x,y)
                break
        return result

    def move(self):
        '''Move to an open, neighboring position '''
        location = self.check_grid(int)
        if location:
                # print('Move, {}, from {},{} to {},{}'.format( \
                #       type(self),self.x,self.y,location[0],location[1]))
            self.island.remove(self)    # remove from current spot
            self.x = location[0]        # new coordinates
            self.y = location[1]
            self.island.register(self) # register new coordinates
```

Back to Breeding

Now we have a method that can check the eight neighbors for a location of a particular type. If our instance's local `breed_clock` hits 0, we will look for an open neighbor. Using the `check_grid` method, we will look for a location that contains an *`int`*. If one is found, we can initialize the breed process. That requires the following steps:

- We must reset the instance's `breed_clock` to the standard `breed_time` contained in the class so it may breed again.
- We must create a new instance (for the offspring) with the proper x and y values—from the open neighboring location.
- We must register the new offspring instances with the `Island` instance.

Interestingly, if this is an `Animal` method, how do we know which kind of instance to make, a `Predator` or a `Prey`? We want the method to be flexible so it will make either under the correct circumstances. Usefully, every instance knows the class to which it belongs, stored in the special variable `__class__`. We capture that class in the

the_class variable, then call the constructor on that class. The breed method is shown in Code Listing 13.11.

Code Listing 13.11

```
def breed(self):
    ''' Breed a new Animal.  If there is room in one of the 8 locations,
    place the new Prey there. Otherwise, you have to wait.
    '''
    if self.breed_clock <= 0:
        location = self.check_grid(int)
        if location:
            self.breed_clock = self.breed_time
            the_class = self.__class__
            new_animal = the_class(self.island,x=location[0],y=location[1])
            self.island.register(new_animal)
```

13.4.3 Eating

Interestingly, the eat method has much in common with both the move and breed methods given the existence of the check_grid method. There are some differences, though. This eating method *must* be in the Predator class, as Prey instances are not allowed to eat. Further, there is a little more manipulation that must be done to deal with eating.

Remember the rules. A Predator instance can look in the local neighborhood for a Prey instance. If one is found, the Predator instance moves to that location and removes the Prey instance from the Island instance. In particular, the eating method must do the following:

- Call the check_grid method passing the Prey class as the type_looking_for argument.
- If a location is found with a Prey instance:
  - Remove the Prey instance from the island.
  - Remove the Predator instance—the instance indicated by the self variable, from the Island instance.
  - Update the Predator instance to the new *x* and *y* coordinates, the coordinates of the Prey that was just removed.
  - Re-register the Predator instance, now at the new location.
  - Set the starve_clock of the Predator instance to the original Predator.starve_time value (so it will not starve and will continue to operate).
- If no Prey instance is found, the method does nothing.

The code for eating is shown in Code Listing 13.12.

Code Listing 13.12

```
    def eat(self):
        ''' Predator looks for one of the eight locations with Prey. If found,
        moves to that location, updates the starve clock, removes the Prey.
        '''
        location = self.check_grid(Prey)
        if location:
            self.island.remove(self.island.animal(location[0],location[1]))
            self.island.remove(self)
            self.x=location[0]
            self.y=location[1]
            self.island.register(self)
            self.starve_clock=self.starve_time
```

13.4.4 The Tick of the Clock

Every pass through the event loop should update each instance's present status. In particular, the individual clocks that were initialized to their class values (breed and starve times) must be decremented. Therefore, we need a method for each class that updates the internal instance clocks on each tick of the clock. We create two methods in each subclass, called `clock_tick`.

The `Prey clock_tick` decrements the instance's `breed_clock`. The `Predator clock_tick` method decrements its `breed_clock`, but it also checks to see whether the `starve_clock` has hit 0. If so, then the instance has starved and must be removed.

The two methods are shown in Code Listing 13.13:

Code Listing 13.13

```
class Prey(Animal):
    # ...
    def clock_tick(self):
        '''Prey updates only its local breed clock
        '''
        self.breed_clock -= 1

class Predator(Animal):
    # ...
    def clock_tick(self):
```

```
        ''' Predator updates both breeding and starving
        '''
        self.breed_clock -= 1
        self.starve_clock -= 1
        if self.starve_clock <= 0:
            self.island.remove(self)
```

13.5 REFINEMENT: HOW MANY TIMES TO MOVE?

Remember our picture of the eight neighbors available for a local move? (Figure 13.2)

Our present event loop does the following:

- It moves through the `Island` instance a row a time, through each *y* value.
- If there is an animal instance in that row (the *x* value) and *y* value, it gives that animal an opportunity to:
  - Potentially eat, if it is a `Predator` instance.
  - Move if it can
  - Breed if it can
  - Do whatever must be done to the instance every clock tick

If our animal moves up or to the left, things seem to work fine. But what happens if our animal moves down or to the right? In those cases, either *x* is incremented (moving down) or *y* is incremented (moving right). Lets look at the "moving down" case. From wherever the instance starts, it moves to a new location with an *x* value one greater. The event loop processes *x* and *y* values by incrementing first all the *y* values in a row, then incrementing the *x* value to begin a new row. This ordering means that the instance we just moved will be given the opportunity to move again because it moved into the next row. If it moves down again, it will get yet another chance to move. Thus, a single instance may move multiple times during the same clock tick. A `Predator` instance could similarly eat multiple times, if—every time it eats—it eats going down or to the right.

The result may or may not be the behavior we wanted, but in truth it makes sense to have one move (eat) per clock tick per instance. Each instance should move (eat) at most one time-event clock tick. The problem is really one of isolating the global state, the grid of `Predator` and `Prey` instances, in time. Each clock tick should have its own global state, but in the implementation we present, we have a common global state across all ticks of the clock.

How can we repair this? There are a number of approaches we might take. We could:

- Have *two* grids, one for the present clock tick and one for the next. Moves would be from the location in the present grid to the future grid of the next clock tick, as would

eating. When the present clock tick ends the new grid becomes the present grid and a fresh empty grid, created for the next clock tick.
- Create a list of instances from the global state and move only instances from the list. Once moved, they are removed from the list and not moved again during this clock tick.
- Mark each instance with a flag, indicating movement. During the present clock tick, if an instance is moved, its flag is changed. All of the movement methods pay attention to the flag and will not move it again if the flag is set. All the movement flags must be cleared on every instance after the present clock tick.

Each approach has its advantages and disadvantages. The multiple-grid approach is more realistic, and more in tune with having multiple global states for different time periods. However, the changes to the present code to include multiple grids would be extensive. Moreover, instances that cannot move must still be copied into the new grid. The list approach is easy for movement, but complicated for eating. If a `Predator` instance eats a `Prey` instance, we have to search through the list to make sure it is also removed. Marking is conceptually simple and requires minimal code updates, but it is time consuming. We have to manipulate all the instances on every cycle to clear their move flags.

Ultimately, we chose the marking approach because it fit best with our existing design. To implement it, we need to make changes in the following locations:

- The `Animal __init__` method must add a new variable `moved` to every instance and initialize it to `False` (the instance has not been moved).
- The `move` and `eat` methods must be modified to check whether the instance has its moved flag set to `True`. If so, it should not be moved again. If the flag is not set, the instance can be moved and its `moved` flag should be set to `True` (so it cannot move again during this clock tick).
- The `Island` class gains a new method, `clear_all_moved_flags`. This method clears the `moved` flag of every instance on the island to `False`.
- The main loop must call the `clear_all_moved_flags` at the beginning of every clock tick.



VISUAL VIGNETTE

13.6 GRAPHING POPULATION SIZE

One of the original ideas of predator–prey simulations is to follow the change in population size during the course of the simulation. Watching the print outs of the `Island` instance does not convey that well. It would be useful to add graphing to observe the changes. Pylab to the rescue!

In this situation, it would be useful to plot both the `Predator` and `Prey` population counts during the course of the run. Pylab works best by collecting the *y* values of each